PRACTICAL 5

Design and Develop an App for Device Group Messaging in Ubiquitous Computing

1. Aim

To design and develop an Android application that allows multiple devices/users to join a common group and exchange messages in real time using Firebase Realtime Database.

2. Software and Requirements

Software Used

- Android Studio Quail 1
- Java programming language
- Firebase
- Firebase Realtime Database
- Android Emulator or Android Mobile Device
- Internet connection

Hardware Required

- Computer/Laptop
- Android device or emulator
- Internet connection

3. Basic Idea of the Application

The application works as follows:

```
User opens the app
        ↓
Enters Name
        ↓
Enters Group Name
        ↓
Clicks JOIN GROUP
        ↓
Enters the Group Chat
        ↓
Sends a Message
        ↓
Message is stored in Firebase
        ↓
Other devices in the same group receive the message
```

Example

Device 1:

```
Name: Harshi
Group: BCA
```

Device 2:

```
Name: Rahul
Group: BCA
```

Both devices join the same BCA group and can see each other's messages.

If another user enters:

```
Name: Priya
Group: CSE
```

Priya joins a separate CSE group.

So Firebase stores data approximately like:

```
groups
│
├── BCA
│    ├── message1
│    │    ├── sender: Harshi
│    │    └── text: Hello
│    │
│    └── message2
│         ├── sender: Rahul
│         └── text: Hi
│
└── CSE
     └── message1
          ├── sender: Priya
          └── text: Hello CSE
```

PART A — CREATE THE ANDROID PROJECT

Step 1: Open Android Studio

Open Android Studio Quail 1.

Click:

```
New Project
```

Step 2: Select Project Type

Select:

```
Phone and Tablet
        ↓
Empty Views Activity
```

Click:

```
Next
```

Step 3: Enter Project Details

Enter the following details:

```
Name: GroupMessaging
```

```
Package name: com.example.groupmessaging
Language: Java
```

Select a suitable minimum SDK, for example:

```
API 24 or above
```

Then click:

```
Finish
```

Wait until Android Studio creates the project and completes the initial Gradle sync.

PART B — CREATE A FIREBASE PROJECT

Step 4: Open Firebase

Open the Firebase Console and sign in using a Google account.

Create a new project.

Click:

```
Create a project
```

Enter the project name:

```
GroupMessaging
```

Then continue through the Firebase project creation steps.

Step 5: Add an Android App to Firebase

Inside the Firebase project, click:

```
Add app
```

Select the:

```
Android icon
```

Firebase will ask for the Android package name.

Enter exactly:

```
com.example.groupmessaging
```

⚠️ The package name in Firebase should match the package/application ID used by the Android project.

Then click:

```
Register app
```

PART C — DOWNLOAD google-services.json

Step 6: Download the Firebase Configuration File

Firebase will show:

```
Download google-services.json
```

Click it to download the file.

Step 7: Add the File to Android Studio

In Android Studio, place the downloaded file inside the app folder.

The correct location is:

```
GroupMessaging
│
├── app
│   ├── google-services.json   ← HERE
│   ├── src
│   └── build.gradle.kts
│
└── gradle
```

Do not place it inside:

```
java
res
layout
```

It must be directly inside the app module.

PART D — ADD FIREBASE DEPENDENCIES

Android Studio Quail 1 uses a Version Catalog, so we edit:

```
Gradle Scripts
    ↓
libs.versions.toml
```

Step 8: Open libs.versions.toml

Go to:

```
Gradle Scripts
    ↓
libs.versions.toml
```

Your existing file contains sections such as:

```
[versions]
[libraries]
[plugins]
```

Add the Google Services version under [versions]:

```
googleServices = "4.4.4"
```

Then under [plugins], add:

```
google-services = { id = "com.google.gms.google-services", version.ref = "googleServices" }
```

The important final sections are:

```
[versions]
agp = "9.2.1"
googleServices = "4.4.4"
junit = "4.13.2"
junitVersion = "1.3.0"
espressoCore = "3.7.0"
```

```
appcompat = "1.8.0"
material = "1.14.0"
activityKtx = "1.13.0"
constraintlayout = "2.2.2"

[libraries]
junit = { group = "junit", name = "junit", version.ref = "junit" }
ext-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
"espressoCore" }
appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
material = { group = "com.google.android.material", name = "material", version.ref =
"material" }
activity-ktx = { group = "androidx.activity", name = "activity-ktx", version.ref =
"activityKtx" }
constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
google-services = { id = "com.google.gms.google-services", version.ref = "googleServices" }
```

PART E — ADD THE GOOGLE SERVICES PLUGIN

Step 9: Open Module-Level build.gradle.kts

In Android Studio, go to:

```
Gradle Scripts
    ↓
build.gradle.kts (Module: app)
```

At the top, use:

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.google.services)
}
```

Step 10: Add Firebase Database Dependency

In the same file, find:

```
dependencies {
```

Add:

```
// Firebase
implementation(platform("com.google.firebase:firebase-bom:34.18.0"))
implementation("com.google.firebase:firebase-database")
```

So the dependency section becomes:

```
dependencies {
    implementation(libs.activity.ktx)
    implementation(libs.appcompat)
    implementation(libs.constraintlayout)
    implementation(libs.material)
```

```
    // Firebase
    implementation(platform("com.google.firebase:firebase-bom:34.18.0"))
    implementation("com.google.firebase:firebase-database")

    testImplementation(libs.junit)
    androidTestImplementation(libs.espresso.core)
    androidTestImplementation(libs.ext.junit)
}
```

Then click:

```
Sync Now
```

Wait until Gradle synchronization completes successfully.

PART F — CREATE FIREBASE REALTIME DATABASE

Step 11: Open the Firebase Menu

In Firebase Console, open the left navigation menu.

Click:

```
Databases and storage
        ↓
Realtime Database
```

Step 12: Create Database

Click:

```
Create Database
```

Choose the recommended/default database location.

Click:

```
Next
```

Step 13: Select Test Mode

For this simple practical, select:

```
Start in test mode
```

Then click:

```
Enable
```

The database will be created.

Initially, the Data section may show:

```
null
```

This is normal because no messages have been sent yet.

The current temporary rules may look similar to:

```
{
  "rules": {
```

```
    ".read": "now < 1790274600000",
    ".write": "now < 1790274600000"
  }
}
```

This allows temporary read and write access until the specified expiry date.

PART G — CREATE THE MAIN SCREEN

The first screen asks the user for:

- Name
- Group Name
- Join Group button

Step 14: Open activity_main.xml

Go to:

```
app
    ↓
res
    ↓
layout
    ↓
activity_main.xml
```

If the file opens in Design mode, click:

```
Code
```

Replace the complete existing code with:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Device Group Messaging"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="30dp" />

    <EditText
        android:id="@+id/etName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter your name"
        android:inputType="text" />

    <EditText
        android:id="@+id/etGroup"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:hint="Enter group name"
        android:inputType="text"
        android:layout_marginTop="15dp" />

    <Button
        android:id="@+id/btnJoin"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="JOIN GROUP"
        android:layout_marginTop="20dp" />

</LinearLayout>
```

PART H — ADD MAIN ACTIVITY CODE

Step 15: Open MainActivity.java

Go to:

```
app
    ↓
kotlin+java
    ↓
com.example.groupmessaging
    ↓
MainActivity.java
```

Replace the existing code with:

```
package com.example.groupmessaging;

import android.content.Intent;
import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {

    EditText etName, etGroup;
    Button btnJoin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_main);

        // Connect Java variables with XML views
        etName = findViewById(R.id.etName);
        etGroup = findViewById(R.id.etGroup);
        btnJoin = findViewById(R.id.btnJoin);

        // Join group button
        btnJoin.setOnClickListener(v -> {
```

```
            String name = etName.getText().toString().trim();
            String group = etGroup.getText().toString().trim();

            // Check if fields are empty
            if (name.isEmpty() || group.isEmpty()) {

                Toast.makeText(MainActivity.this,
                        "Please enter name and group name",
                        Toast.LENGTH_SHORT).show();

            } else {

                // Open chat screen
                Intent intent = new Intent(
                        MainActivity.this,
                        chatActivity.class
                );

                // Send name and group to next screen
                intent.putExtra("name", name);
                intent.putExtra("group", group);

                startActivity(intent);
            }
        });
    }
}
```

PART I — CREATE THE CHAT ACTIVITY

Step 16: Create a Java Class

Right-click the package:

```
com.example.groupmessaging
```

Then select:

```
New
    ↓
Java Class
```

Enter:

```
chatActivity
```

Then click:

```
OK
```

Your project will contain:

```
MainActivity.java
chatActivity.java
```

⚠️ Use the same capitalization consistently. Your uploaded project currently uses chatActivity with a lowercase c, so the practical above follows your actual project.

PART J — CREATE THE MESSAGE CLASS

Step 17: Create Message.java

Again, right-click:

```
com.example.groupmessaging
```

Then select:

```
New
    ↓
Java Class
```

Enter:

```
Message
```

Click OK.

Replace its code with:

```
package com.example.groupmessaging;

public class Message {

    public String sender;
    public String text;

    // Empty constructor required by Firebase
    public Message() {
    }

    // Constructor for creating a message
    public Message(String sender, String text) {
        this.sender = sender;
        this.text = text;
    }
}
```

Purpose

This class stores each message with:

- Sender Name
- Message Text

For example:

```
sender: Harshi
text: Hello everyone!
```

PART K — CREATE THE CHAT LAYOUT

Step 18: Create activity_chat.xml

Go to:

```
app
    ↓
res
    ↓
layout
```

Right-click:

```
layout
```

Then select:

```
New
    ↓
Layout Resource File
```

Enter:

```
File name: activity_chat
Root element: LinearLayout
```

Click:

```
OK
```

Step 19: Add Chat Screen XML

Open:

```
activity_chat.xml
```

Replace the complete code with:

```
<?xml version="1.0" encoding="utf-8"?>

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="12dp">

    <!-- Group Name -->
    <TextView
        android:id="@+id/tvGroup"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Group Chat"
        android:textSize="22sp"
        android:textStyle="bold"
        android:gravity="center"
        android:padding="10dp" />

    <!-- Messages -->
    <TextView
        android:id="@+id/tvMessages"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="No messages yet"
        android:textSize="17sp"
        android:padding="10dp"
        android:background="#EEEEEE" />

    <!-- Bottom message area -->
    <LinearLayout
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginTop="10dp">

        <EditText
            android:id="@+id/etMessage"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:hint="Type message"
            android:inputType="text" />

        <Button
            android:id="@+id/btnSend"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="SEND" />

    </LinearLayout>

</LinearLayout>
```

The screen will approximately contain:

```
          Group: BCA

Harshi: Hello
Rahul: Hi


[ Type message             ][SEND]
```

PART L — ADD CHAT ACTIVITY CODE

Step 20: Open chatActivity.java

Replace its code with:

```
package com.example.groupmessaging;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import com.google.firebase.database.DataSnapshot;
import com.google.firebase.database.DatabaseError;
import com.google.firebase.database.DatabaseReference;
import com.google.firebase.database.FirebaseDatabase;
import com.google.firebase.database.ValueEventListener;

public class chatActivity extends AppCompatActivity {

    TextView tvGroup, tvMessages;
    EditText etMessage;
```

```
    Button btnSend;

    String name, group;

    DatabaseReference databaseReference;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_chat);

        // Connect Java code with XML views
        tvGroup = findViewById(R.id.tvGroup);
        tvMessages = findViewById(R.id.tvMessages);
        etMessage = findViewById(R.id.etMessage);
        btnSend = findViewById(R.id.btnSend);

        // Get name and group from MainActivity
        name = getIntent().getStringExtra("name");
        group = getIntent().getStringExtra("group");

        tvGroup.setText("Group: " + group);

        // Firebase path for this group
        databaseReference = FirebaseDatabase.getInstance()
                .getReference("groups")
                .child(group);

        // Send message
        btnSend.setOnClickListener(v -> sendMessage());

        // Receive messages
        receiveMessages();
    }

    private void sendMessage() {

        String text = etMessage.getText().toString().trim();

        if (!text.isEmpty()) {

            String id = databaseReference.push().getKey();

            Message message = new Message(name, text);

            if (id != null) {
                databaseReference.child(id).setValue(message);
            }

            etMessage.setText("");
        }
    }

    private void receiveMessages() {

        databaseReference.addValueEventListener(
                new ValueEventListener() {
```

```
                    @Override
                    public void onDataChange(@NonNull DataSnapshot snapshot) {

                        String allMessages = "";

                        for (DataSnapshot dataSnapshot :
                                snapshot.getChildren()) {

                            Message message =
                                    dataSnapshot.getValue(Message.class);

                            if (message != null) {
                                allMessages += message.sender + ": "
                                        + message.text + "\n\n";
                            }
                        }

                        tvMessages.setText(allMessages);
                    }

                    @Override
                    public void onCancelled(
                            @NonNull DatabaseError error) {
                    }
                });
    }
}
```

PART M — UPDATE ANDROID MANIFEST

Step 21: Open AndroidManifest.xml

Go to:

```
app
    ↓
manifests
    ↓
AndroidManifest.xml
```

Use:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- Internet permission for Firebase -->
    <uses-permission android:name="android.permission.INTERNET" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.GroupMessaging">
```

```
        <!-- Chat screen -->
        <activity
            android:name=".chatActivity"
            android:exported="false" />

        <!-- Main screen -->
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:windowSoftInputMode="adjustResize">

            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>

        </activity>

    </application>

</manifest>
```

PART N — PROJECT STRUCTURE

After completing all steps, your important project structure should be:

```
GroupMessaging
│
├── app
│   ├── google-services.json
│   ├── build.gradle.kts
│   │
│   └── src
│       └── main
│           │
│           ├── java
│           │   └── com.example.groupmessaging
│           │       ├── MainActivity.java
│           │       ├── chatActivity.java
│           │       └── Message.java
│           │
│           ├── res
│           │   └── layout
│           │       ├── activity_main.xml
│           │       └── activity_chat.xml
│           │
│           └── AndroidManifest.xml
│
└── gradle
    └── libs.versions.toml
```

PART O — RUN THE APPLICATION

Step 22: Connect a Device or Start Emulator

At the top of Android Studio:

- Select an Android device or emulator.
- Click the green Run ▶ button.

Wait for the application to install.

PART P — TEST THE APPLICATION

Test 1: Join a Group

On Device 1, enter:

```
Name: Harshi
Group Name: BCA
```

Click:

```
JOIN GROUP
```

The chat screen opens.

Send:

```
Hello everyone!
```

Test 2: Use Another Device

Run the same application on another device/emulator.

Enter:

```
Name: Rahul
Group Name: BCA
```

Click:

```
JOIN GROUP
```

Both users are in the same group.

When Harshi sends a message, Rahul can receive it.

When Rahul sends a message, Harshi can receive it.

This demonstrates real-time group messaging between devices.

PART Q — HOW MULTIPLE GROUPS WORK

There is no fixed number of groups in this simple application.

For example:

```
Group: BCA
Group: CSE
Group: Teachers
Group: Students
Group: FYBCA
```

Each different group name creates a different location in Firebase.

The important code is:

```
databaseReference = FirebaseDatabase.getInstance()
        .getReference("groups")
        .child(group);
```

Explanation

If:

```
group = BCA
```

Firebase path becomes:

```
groups/BCA
```

If:

```
group = CSE
```

Firebase path becomes:

```
groups/CSE
```

Therefore, each group has separate messages.

PART R — HOW TO GO BACK AND JOIN ANOTHER GROUP

The simplest method is to use the Android device's Back button.

```
Chat Screen
    ↓
Press Back
    ↓
Main Screen
    ↓
Enter another Group Name
    ↓
JOIN GROUP
```

For example:

```
Current Group: BCA
       ↓
Press Back
       ↓
Enter Group: CSE
       ↓
JOIN GROUP
```

PART S — WORKING OF THE APPLICATION

The application works in the following sequence:

```
1. User enters name
        ↓
2. User enters group name
        ↓
3. JOIN GROUP button is clicked
        ↓
4. Name and group are sent to Chat Activity using Intent
        ↓
```

```
5. Firebase reference is created:
   groups/groupName
        ↓
6. User types a message
        ↓
7. Message is converted into a Message object
        ↓
8. Firebase stores the message
        ↓
9. ValueEventListener detects database changes
        ↓
10. Messages are displayed on all devices
    connected to the same group
```

Algorithm

1. Start the application.
2. Accept the user's name and group name.
3. Check whether both fields are filled.
4. Open the chat screen.
5. Receive the name and group name using Intent.
6. Create a Firebase reference for the selected group.
7. Accept the message from the user.
8. Create a unique ID using push().
9. Store the sender name and message in Firebase.
10. Listen continuously for changes in Firebase.
11. Display all messages belonging to that group.
12. Stop.

Result

Thus, an Android application for device group messaging was successfully designed and developed using Java, Android Studio, Firebase, and Firebase Realtime Database. Multiple devices can join the same group and exchange messages in real time. Different group names create separate communication groups.